

CMDA 3634 SP2024 Parallel Programming Skills Activity

GPU Standard Deviation

50 points

Instructions

- Complete this Jupyter notebook by writing and testing the requested CUDA code.
- Also, answer the given questions in the markdown cells provided.
- **Upload your completed .ipynb file to your cmda3634_arc repo on code.vt.edu under the directory PSA05.**
- **Also submit a printed version of your .ipynb file as a .pdf file to Canvas.**

Academic Integrity

- The use of code from prior sections of the class (or similar classes at other institutions, **Chegg, Course Hero, GitHub, Stack Overflow, ChatGPT, rent-a-coder sites, etc.**) is **strictly prohibited**, regardless of how they are obtained.

Honor Code

- By submitting this assignment, you acknowledge that you have adhered to the Virginia Tech Honor Code and attest to the following:

I have neither given nor received unauthorized assistance on this assignment. The work I am presenting is ultimately my own.

✓ Standard Deviation

- The standard deviation of the numbers $1 \dots N$ is given by

$$\sigma = \sqrt{\frac{N^2 - 1}{12}}$$

- A sequential code to compute the standard deviation of the numbers $1 \dots N$ is given below.
- To test the sequential version, configure Google Colab to use a CPU runtime (not GPU!).

```

%%writefile std_dev.c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef unsigned long long int uint64;

int main (int argc, char** argv) {

    /* get N from the command line */
    if (argc < 2) {
        printf ("Command usage : %s %s\n",argv[0], "N");
        return 1;
    }
    uint64 N = atol(argv[1]);

    /* compute the mean */
    uint64 sum = 0;
    for (uint64 i=1;i<=N;i++) {
        sum += i;
    }
    double mean = 1.0*sum/N;

    /* compute the sum of differences squared */
    double sum_diff_sq = 0;
    for (uint64 i=1;i<=N;i++) {
        sum_diff_sq += (i-mean)*(i-mean);
    }

    /* compute the standard deviation */
    double std_dev = sqrt(sum_diff_sq/N);

    /* print the results */
    printf ("computed std dev is %.11f",std_dev);
    printf (" , sqrt((N^2-1)/12) is %.11f\n",sqrt((N*N-1)/12.0));

}

```

Overwriting std_dev.c

```
!gcc -o std_dev std_dev.c -lm
```

```
!time ./std_dev 100000000
```

computed std dev is 28867513.5, sqrt((N^2-1)/12) is 28867513.5

```

real    0m0.687s
user    0m0.646s
sys     0m0.007s

```

✓ Part 1 : Complete a CUDA standard deviation version that uses a single thread block.

25 points

For this part the main function is already complete so you just need to write the kernel.

You will need to have one thread print the standard deviation from inside the kernel.

Use a GPU runtime on Google Colab to test your code.

Be careful to only use the GPU runtime when you are actively running CUDA code.

It is possible to temporarily lose your free access to a GPU on Google!

In particular, when writing code or answering questions you should have your GPU runtime disconnected.

You can disconnect your GPU runtime by selecting *Disconnect and delete runtime* from the *Runtime* menu.

Question: Explain how you made your kernel code thread safe and parallel efficient.

Answer: The first thing that was done is creating the necessary local variables for thread. This being both `thread_num` and `num_threads`. I then had to create my shared variables which were for the sum and `sum_diff_sq`. Next, and I do believe this was important, it was to add an initialize condition as to when to set our shared variables to 0. This was not only the important part, we also had to sync our threads

to make sure that they were both at the barrier. After this we had to create variables to iterate through our for-loops. This is where we needed to begin at `thread_num+1` and increment by `num_threads`. We had to add this to `thread_sum` because after we had to add it to our actual shared sum. Once we ran the for-loop we had to call the function `atomicAdd` to add all the values of `thread_sum` and put it in the sum address. Finally, we also needed to check if our threads have reached the barrier/are all in sync. Finally, we need to make our `sum_diff_sq` parallel. Here we make our for-loop parallel in the same way. However, we did need to calculate the mean by using the final sum value and dividing by the total number of values. Once we probably adjusted our for-loop and added it to our local `thread_sum_diff` variable, we add this to our actual `sum_diff_sq` and check for our threads to be insync. The few key important parts for making this code thread safe and parallel are ensuring that we correctly parallelizing our for-loop by flattening our data. Correctly using the appropriate communicative functions such as `atomic` `[Name]([pointer],[variable])`. Finally, the last part that is important is to make sure our code is running through the entire thread count. This means for our to sync our threads i.e. our threads reach/hit the barrier. So that all threads are run and we are not skipping or not finishing it correctly.

Hints: Your kernel should use 3 barriers. How many times does each thread execute an atomic instruction?

Question: What is the primary limitation of using a single thread block? Explain in terms of the GPU hardware (i.e. the SMs--symmetric multiprocessors)

Answer: It cannot fully utilize the available resources of the Streaming Multiprocessors (SM's). Each SM on a GPU is able to run multiple threads consecutively. If we use a single thread block it might not be enough parallelism to fully use all the available SM's on the GPU.

```
%%writefile gpu_std_dev_v1.cu
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

typedef unsigned long long int uint64;

__global__ void stdevKernel(uint64 N)
{
    // initializing our CUDA thread variables
    int thread_num = threadIdx.x;
    int num_threads = blockDim.x;

    // shared vairable
    __shared__ uint64 sum;
    __shared__ double sum_diff_sq;

    // initialize share variables
    if (thread_num == 0)
    {
        sum = 0;
        sum_diff_sq = 0.0;
    }
    __syncthreads();

    uint64 thread_sum = 0;
    for(int i = 1+thread_num; i<= N; i+=num_threads)
    {
        thread_sum += i;
    }
    atomicAdd(&sum, thread_sum);
    __syncthreads();

    double mean = (1.0)*sum/N;

    double thread_sum_diff_sq = 0.0;
    for (int i = 1+thread_num; i <= N; i+=num_threads)
    {
        thread_sum_diff_sq += (i-mean)*(i-mean);
    }
    atomicAdd(&sum_diff_sq, thread_sum_diff_sq);
    __syncthreads();

    if (thread_num == 0)
    {
        // printf("standard deviation is %.4f\n",sqrt(sum_diff_sq/N));
        printf ("computed std dev is %.11f",sqrt(sum_diff_sq/N));
        printf ("sqrt((N^2-1)/12) is %.11f\n",sqrt((N*N-1)/12.0));
    }
}
```

```

}

int main(int argc, char **argv)
{
    /* get N and num_threads from the command line */
    if (argc < 3)
    {
        printf ("Command usage : %s %s %s\n",argv[0],"N","num_threads");
        return 1;
    }

    uint64 N = atol(argv[1]);
    int num_threads = atoi(argv[2]);

    printf ("num_threads = %d\n",num_threads);

    stdevKernel <<< 1, num_threads >>> (N);
    cudaDeviceSynchronize();
}

Writing gpu_std_dev_v1.cu

```

```
!nvcc -arch=sm_75 -o gpu_std_dev_v1 gpu_std_dev_v1.cu
```

✓ Use N equal to 1 billion to test your code for accuracy.

Note: Here we are using 128 threads.

```
!time ./gpu_std_dev_v1 1000000000 128

num_threads = 128
computed std dev is 288675134.6, sqrt((N^2-1)/12) is 288675134.6

real    0m1.644s
user    0m1.307s
sys     0m0.236s

```

✓ Use N equal to 10 billion to test your code for speed.

Note: In order to load the GPU we need to use a large value of N . Unfortunately, the intermediate steps in the standard deviation calculation overflow the double precision data type and so the answers output are not correct!

```
!time ./gpu_std_dev_v1 10000000000 128
```

```
num_threads = 128
computed std dev is 484971221.4, sqrt((N^2-1)/12) is 804481180.2
```

```
real    0m2.926s
user    0m2.696s
sys     0m0.217s
```

✓ Part 2 : Complete a CUDA Standard Deviation Version that Uses multiple thread blocks.

25 points

For this part you will need to write two kernels. The interfaces to the kernels are provided below.

Note: In each kernel, each thread calculates the sum of T terms.

In addition you will have to finish writing the main function which is partially provided below.

Question: Explain how you made your kernel code thread safe and parallel efficient.

Answer: I used the example from lecture 26 on how to make this code parallel and thread safe. For this, I used the same example to declare the thread_num by using the equation

$blockID \cdot blockDim + threadID$. We then initialized start and end with $thread_num \cdot T$ and $start + T$ for both the sum and sumdiffsq Kernel. For

both we also want to check if end is greater than N (the size of data). Now for each of these we parallelize our for-loop by starting at $1 + \text{start}$ and incrementing by one until it is less than or equal to end. For the respective functions, sum is just added up from i and then we put it into function `atomicAdd`. Where sum is a pointer into our function and then add the local variable `thread_sum` so it does not affect our end result. Now for `sumdiffsqKernel()`. we do the same thing by creating a local variable to hold the iterations for $(i - \text{mean})^2$.

Question: About how many times faster is your version 2 kernel (with T equal to 1000) than your version 1 kernel for N equal to 10 billion?

Answer: Okay, so part 1 values are: real 0m2.842s, user 0m2.715s, and sys 0m0.116s. Part 2 values are: real 0m0.614s, user 0m0.433s, and sys 0m0.143s. Besides the fact, that the computed standard deviation is not the same (one would assume) part 1 is most likely incorrect and part 2 is correct. As such part 2 also ran fully through the entire thread without hitting overflow. So when we consider the speed up times for; real: $2.842 / 0.614 = 4.62866$, user: $2.715 / 0.433 = 6.27020785$, and sys: $0.116 / 0.143 = 0.8111888$. So when we evaluate the speedup for real it is about 4.629 times faster.

Question: When you run your version 2 with N equal to 1 billion and T equal to 1, the runtime is actually longer than the version 1 runtime with the same value of N ! Explain why version 2 is slower than version 1 when T is equal to 1

despite the fact that version 2 is using every SM and version 1 is only using 1 SM.

Answer: All parts for N equal to 1 billion values for part 1 are; real 0m1.527s, user 0m1.335s, and sys 0m0.117s. For part 2 values: real 0m3.090s, user 0m2.940s, and sys 0m0.134s. This is because of T, hence, the number of terms per thread. If we increase this, our code would compile quicker. If we were to increase it to oh lets say 1000, we would be able to have more terms per thread (more values to run per thread) which would make our code compile significantly quicker.

```
%%writefile gpu_std_dev_v2.cu
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

typedef unsigned long long int uint64;

__global__ void sumKernel(uint64 N, uint64 T, uint64* sum)
{
    /* copied from lecture 26 */

    uint64 thread_num = (uint64)blockIdx.x*blockDim.x + threadIdx.x;
    uint64 thread_sum = 0;
    uint64 start = thread_num*T;
    uint64 end = start+T;
    if (end > N)
    {
        end = N;
    }

    for (uint64 i = 1+start; i<=end;i++)
    {
        thread_sum += i;
    }
    atomicAdd(sum,thread_sum);
}

__global__ void sumdiffsqKernel(uint64 N, uint64 T, double mean, double* sum_diff_sq)
{
    uint64 thread_num = (uint64)blockIdx.x*blockDim.x + threadIdx.x;
    double thread_sum_diff_sq = 0.0;
    uint64 start = thread_num*T;
    uint64 end = start+T;
    if (end > N)
    {
        end = N;
    }

    for (uint64 i = 1+start; i<=end; i++)
    {
        thread_sum_diff_sq += (i-mean)*(i-mean);
    }
    atomicAdd(sum_diff_sq, thread_sum_diff_sq);
}

int main (int argc, char** argv)
{

    /* get N, T, and B from the command line */
```



```
/* T is the number of terms per thread */
/* B is the number of threads per block */
/* we typically choose B to be a multiple of 32 */
/* the maximum value of B is 1024 */
if (argc < 4)
{
    printf ("Command usage : %s %s %s %s\n",argv[0],"N","T","B");
    return 1;
}

uint64 N = atol(argv[1]);
uint64 T = atol(argv[2]);
int B = atoi(argv[3]);

/* G is the number of thread blocks */
/* the maximum number of thread blocks G is  $2^{31} - 1 = 2147483647$  */
/* We choose G to be the minimum number of thread blocks to have at least N/T threads */

/*****/
/* add your code to compute G here */
int G = (N+(B*T)-1)/(B*T);
/*****/

printf ("N = %lld\n",N);
printf ("terms per thread T = %lld\n",T);
printf ("threads per block B = %d\n",B);
printf ("number of thread blocks G = %d\n",G);
```